AMERICAN UNIVERSITY
OF PHNOM PENH
STUDY LOCALLY. LIVE GLOBALLY.

PhishNet: A Web-Based Phishing Simulation and Security Awareness Training Platform

Securing Human-Centric Attack Surfaces Through Simulation-Based Training

Written by **Savakroth Leav**

Supervised by **Dr. Tek Ming Ng**

Faculty of Digital Technologies

American University of Phnom Penh, Phnom Penh, Cambodia

2023174leav@aupp.edu.kh

Final Year Project — Academic Year 2025/2026

Abstract

Phishing remains the dominant initial access vector in organizational cyber-attacks, responsible for over 90% of recorded data breaches globally. Despite widespread deployment of perimeter security controls, employees continue to fall victim to increasingly sophisticated social engineering campaigns that exploit cognitive biases rather than technical vulnerabilities. This paper presents PhishNet, a comprehensive web-based phishing simulation and security awareness training platform developed as a Final Year Project in Cybersecurity. The platform enables security administrators to design, deploy, and monitor realistic phishing email campaigns targeting registered employees, track all interaction events in real time through four-mechanism instrumentation, and deliver targeted security awareness training content to individuals identified as high-risk through a dynamic scoring model. PhishNet integrates with the GoPhish open-source email delivery framework and supports multiple SMTP sending profiles including sandboxed local testing via Mailhog and real-world delivery via Gmail SMTP. A dynamic risk-scoring model normalized to a 0–100 scale quantifies individual employee susceptibility from five weighted interaction event types. Eight modular training modules with quiz-based assessment and per-user progress persistence deliver measurable knowledge improvement. System evaluation through controlled simulation runs demonstrates accurate end-to-end campaign execution, sub-second tracking event latency, and measurable reductions in employee susceptibility following training completion. PhishNet contributes an accessible, open-architecture alternative to costly commercial platforms, with identified future research directions including transformer-based AI phishing detection and reinforcement learning-driven adaptive training personalization.

Keywords*—phishing simulation; security awareness training; social engineering; GoPhish; risk scoring; Flask; SQLite; web application security; SMTP; email security*

I. INTRODUCTION

Cyber threats that target human behaviors represent one of the most persistent and consequential challenges organizations face across every sector of the global economy. Unlike software vulnerabilities, which can be remediate through systematic patch management and configuration hardening, human susceptibility to social engineering persists as a systemic weakness that is fundamentally independent of the sophistication of deployed technical controls. The 2024 Verizon Data Breach Investigations Report (DBIR) identified phishing as the primary initial access vector in 41% of all confirmed breach incidents analyzed across 30,458 reported security events in 94 countries [1]. The financial consequences are severe: IBM's 2024 Cost of a Data Breach Report documented a global average breach cost of USD 4.88 million, with incidents originating from phishing attacks averaging USD 4.65 million per incident [2].

The scale of the phishing threat continues to accelerate. The Anti-Phishing Working Group (APWG) documented over 1.2 million unique phishing sites in 2023, representing a 150% increase compared to 2020 figures [4]. Business Email Compromise (BEC), a sophisticated spear-phishing variant targeting senior executives—caused an estimated USD 2.9 billion in verified losses reported to the FBI's Internet Crime Complaint Centre (IC3) in 2023 alone [13]. These figures reflect only reported incidents; the actual economic impact of phishing is estimated to be three to five times greater when accounting for unreported losses, productivity costs, and reputational damage.

Simulation-based security awareness training (SAT) has emerged as the most empirically validated human-centric intervention for reducing phishing susceptibility. Research by Proofpoint demonstrates that organizations conducting regular phishing simulations reduce susceptibility rates by up to 72% over a structured 12-month program [3]. The mechanism underlying this effectiveness is the embedded training approach: delivering educational content at the precise moment of a simulated phishing interaction, when the cognitive impact is greatest, maximizes knowledge retention and behavioral modification [6].

Despite this evidence base, organizations seeking to implement phishing simulation programs face a challenging landscape. Commercial platforms such as KnowBe4, serving over 65,000 organizations globally and Proofpoint Security Awareness Training command annual per-seat licensing fees ranging from USD 18 to USD 55 per user for enterprise tiers, rendering them inaccessible to small and medium enterprises, educational institutions, and

organizations in developing economies. Open-source alternatives, principally GoPhish, provide robust email delivery capabilities but lack the integrated training content, employee risk quantification, administrative analytics dashboards, and organizational workflow management that a comprehensive security awareness program requires.

In the Kingdom of Cambodia, the rapid adoption of digital banking and e-government services has outpaced public cybersecurity literacy. As the nation moves toward the goals outlined in the **Cambodia Digital Economy and Society Policy Framework 2021–2035**, the risk of phishing has escalated. Local organizations, particularly Small and Medium Enterprises (SMEs) which constitute over 90% of the Cambodian business landscape, are increasingly targeted by regional 'scam centers' utilizing sophisticated social engineering.

Current solutions in the Cambodian market are bifurcated: expensive international platforms like KnowBe4, which lack Khmer-language support and context-specific templates, or no solution at all. **PhishNet** addresses this by providing a localized, cost-effective alternative that allows Cambodian administrators to simulate threats specific to the local digital ecosystem, such as fraudulent ABA Bank notifications or Telegram-based credential harvesting.

This paper presents PhishNet, a full-stack phishing simulation and security awareness training platform that bridges this gap. The system was developed as a Final Year Project with the objective of delivering enterprise-grade functionality on an open, zero-cost, self-hosted architecture. The primary research and engineering contributions of this work are:

1. A complete end-to-end phishing simulation pipeline integrating GoPhish via its REST API with a custom Python Flask backend, supporting configurable multi-profile SMTP delivery including both sandboxed and real-world email transmission.
2. A quantified dynamic risk-scoring model that computes individual employee susceptibility from five weighted interaction event types, producing a normalized 0–100 risk index that drives targeted training intervention.
3. An integrated eight-module security awareness training system with multiple-choice quiz assessment, per-user local Storage progress persistence, and gamified completion incentives including badge awards and risk score reduction.
4. A production-ready administrative dashboard providing real-time campaign analytics, employee risk visualization, multi-format report export (CSV, JSON, HTML, PDF), and user account management.
5. A configurable Phish Profiles system supporting Mailhog (local testing), Gmail SMTP, Brevo, Outlook, and custom SMTP configurations with an in-application guided setup workflow.

The remainder of this paper is organized as follows. Section II surveys related work in phishing attack methodologies, security awareness research, and existing simulation platforms. Section III presents the system architecture and design methodology. Section IV details the implementation of each subsystem. Section V presents evaluation methodology and results from controlled simulation runs. Section VI discusses system limitations and future research directions. Section VII concludes the paper.

II. LITERATURE REVIEW

A. Phishing Attack Evolution and Taxonomy

Phishing is a category of social engineering attack in which an adversary impersonates a trusted entity—an employer, financial institution, government agency, or technology provider—to deceive a target into disclosing sensitive credentials, executing a malicious payload, or performing an unintended financial action [5]. The technique was first systematically documented in the mid-1990s in the context of America Online (AOL) account theft, but has evolved into a sophisticated, multi-variant attack family that consistently ranks as the most prevalent threat vector in annual breach statistics.

The psychological effectiveness of phishing is grounded in Cialdini's six principles of influence [7]: Authority (emails purporting to originate from IT departments, executives, or government bodies), Urgency (account suspension threats requiring immediate action), Scarcity (limited-time password reset windows), Social Proof (notifications about colleagues' actions), Liking (personalized greetings using harvested personal information), and Reciprocity (offers of gifts or bonuses requiring credential verification). Modern phishing campaigns systematically combine multiple principles in a single

message, reducing target deliberation time and bypassing rational risk assessment.

Contemporary phishing has diversified into a taxonomy of specialized variants. Spear-phishing targets specific individuals using personalized contextual information harvested from social media, LinkedIn profiles, and public records, achieving click rates four to six times higher than generic bulk campaigns [1]. Whaling focuses on C-suite executives and board members, exploiting their organizational authority to approve fraudulent wire transfers. Smishing (SMS-based) and vishing (voice-based) variants extend the attack surface beyond email, with AI-generated voice cloning increasingly rendering traditional caller recognition defenses ineffective. Clone phishing reproduces legitimate prior communications with malicious link substitutions, exploiting established trust relationships with known senders.

B. Security Awareness Training Research

The academic evidence base for simulation-based security awareness training is extensive and consistently positive. Kumaraguru et al. conducted a landmark randomized controlled trial demonstrating that embedded training—delivering educational content immediately following a simulated phishing click—produced 25% higher knowledge retention at 28-day follow-up compared to equivalent content delivered through standalone training sessions conducted independently of simulations [6]. This finding motivates PhishNet's caught-redirect mechanism, which presents contextualized phishing awareness content now of simulated credential submission, maximizing the pedagogical impact of the simulation experience.

A longitudinal study by Benenson et al. tracked phishing susceptibility across 1,200 employees at a German financial services organization across four quarterly simulation cycles [8]. Susceptibility declined from a baseline rate of 45.2% in cycle 1 to 7.8% in cycle 4, a reduction of 83%. However, the study identified a critical operational finding: without ongoing reinforcement, susceptibility rates rebounded to approximately 28% within six months of program cessation. This rebound effect underscores the requirement for sustained, periodic simulation programs rather than one-off interventions, and motivates PhishNet's persistent campaign management architecture supporting indefinitely repeating campaign schedules.

Vishwanath et al. identified individual difference variables that moderate phishing susceptibility, finding that habitual email checking behavior, impulsivity, and low technical self-efficacy were the strongest predictors of click behavior [12]. These findings support the risk-stratified intervention approach implemented in PhishNet, where high-risk employees are prioritized for mandatory training module completion based on their dynamic risk score rather than receiving undifferentiated training alongside the entire employee population.

Research on training content format found that interactive, scenario-based modules produced significantly higher knowledge gains than passive reading materials [6]. PhishNet implements scenario-anchored module content followed by multiple-choice assessment, aligning with the evidence-based format recommendations from the SAT literature.

C. Existing Phishing Simulation Platforms

GoPhish [9] is an open-source phishing simulation framework that provides REST API-based campaign orchestration, email template authoring with a WYSIWYG editor, landing page construction, and per-recipient result tracking through a web-based dashboard. GoPhish provides a robust and battle-tested email delivery engine that handles personalization variable substitution, open-tracking pixel embedding, and click redirect management. However, GoPhish is intentionally minimal in application scope: it provides no training content delivery, no employee risk quantification beyond binary sent/clicked/submitted/reported indicators, no organizational workflow management, and no analytics beyond per-campaign summary tables. PhishNet builds upon GoPhish as its delivery backend, retaining its proven infrastructure while providing the comprehensive application layer that transforms point-in-time simulation data into an ongoing organizational security awareness program.

KnowBe4 [10] is the global market leader in commercial security awareness training, serving over 65,000 customer organizations with a library exceeding 10,000 phishing templates and thousands of training content assets across 34 languages. Its AI-driven phishing difficulty adjustment, automated learning path assignment, and extensive compliance reporting capabilities represent the commercial state of the art. Proofpoint Security Awareness Training offers comparable enterprise capabilities with stronger integration into Proofpoint's email security

gateway for real-threat-to-simulation escalation. Both platforms command annual per-seat pricing that is prohibitive for the majority of organizations outside large enterprises, and their closed architecture prevents the contextual customization required for institution-specific or region-specific simulation scenarios. PhishNet addresses both barriers through its open-architecture, self-hosted design.

Lucy Security and Sophos Phish Threat occupy the mid-market, offering more accessible pricing but with reduced feature depth compared to KnowBe4. Academic literature identifies a consistent gap across all available platforms: none provide open, extensible architectures that allow researchers and practitioners to modify risk-scoring models, integrate custom training content pipelines, or extend the analytics layer—capabilities that PhishNet specifically addresses.

D. Risk Quantification in Security Behavior Research

Sasse et al. proposed a foundational behavioral risk model for information security in which individual risk score is a function of the frequency, recency, and severity of security-relevant actions, arguing that quantified risk metrics enable targeted, proportionate intervention rather than uniform policy enforcement [11]. Adams and Sasse extended this framework to identify three categories of security behavior: compliant (following security policy), non-compliant (violating policy while aware of it), and unaware (violating policy through ignorance), each requiring different intervention types. PhishNet's risk-scoring model operationalizes the Sasse framework for the specific event types instrumentable in a web-based simulation environment, with score deltas calibrated to reflect the relative severity of each event type with respect to actual security compromise probability.

Ryan et al. demonstrated in a field study of 847 employees that risk score feedback provided to employees—rather than only to administrators—produced a 31% greater reduction in susceptibility over a 90-day period compared to a control group receiving only training content without score visibility [12]. This finding motivates PhishNet's employee-facing risk score display in the awareness training interface, which presents the employee's current risk level and module completion progress as motivational feedback.

E. Risk Quantification in Security Behavior Research

Recent reports from the Cambodian Computer Emergency Response Team (CamCERT) highlight a surge in impersonation attacks. Unlike Western contexts where 'Email' is the primary vector, Cambodian users are uniquely vulnerable to 'Hybrid Phishing' were emails lead to Telegram or WhatsApp bots. Furthermore, the linguistic nuances of Khmer require simulation platforms that support Unicode-compliant templates. Existing research often overlooks the 'Trust Culture' prevalent in Southeast Asian organizational hierarchies, where an email from a 'Director' (Authority Principle) is rarely questioned by subordinates, making simulation-based training even more vital for the local workforce.

III. SYSTEM ARCHITECTURE AND METHODOLOGY

A. Requirements Analysis

A structured requirements analysis was conducted to define the functional and non-functional requirements for PhishNet prior to architecture and design. Functional requirements were derived from a review of the security awareness literature, analysis of GoPhish's capability gaps, and evaluation of commercial platform feature sets. Non-functional requirements addressed deploy ability, performance, security, and maintainability constraints appropriate for a single-organization self-hosted deployment.

Key functional requirements identified included: (FR1) Ability to create, configure, and launch phishing email campaigns using customizable templates; (FR2) Automated personalization of phishing emails with per-recipient variables including name, email address, and unique tracking URLs; (FR3) Real-time tracking of four interaction event types: email open, link click, credential submission, and phishing report; (FR4) Dynamic per-employee risk score computation from tracked events; (FR5) Delivery of modular security awareness training content with quiz assessment; (FR6) Administrative dashboard with real-time analytics and campaign result visualisation; (FR7) Multi-format report export; (FR8) Configurable multi-profile SMTP sending with support for local sandbox and real-world delivery; (FR9) Localization and Multi-lingual Support: Ability to render Khmer Unicode characters in email templates and landing pages to reflect the actual phishing lures seen in the Cambodian digital space.

Non-functional requirements included: (NFR1) Deployable on a single commodity Linux machine without cloud infrastructure; (NFR2) API response latency below 500ms for all read operations; (NFR3) Tracking event processing latency below 200ms under concurrent load; (NFR4) Role-based access control preventing employee access to administrative interfaces; (NFR5) Support for a minimum of 500 active campaign targets without performance degradation; (NFR6) Resource Efficiency: Designed to run on low-cost, local server hardware or basic VPS instances, catering to the budget constraints of Cambodian NGOs and educational institutions.

B. System Architecture Overview

PhishNet adopts a three-tier architecture comprising a Vanilla JavaScript frontend, a Python Flask REST API backend, and a SQLite relational database. External integrations include the GoPhish framework for email delivery orchestration and Mailhog for sandboxed SMTP testing. Figure 1 illustrates the overall system architecture and component interaction model.

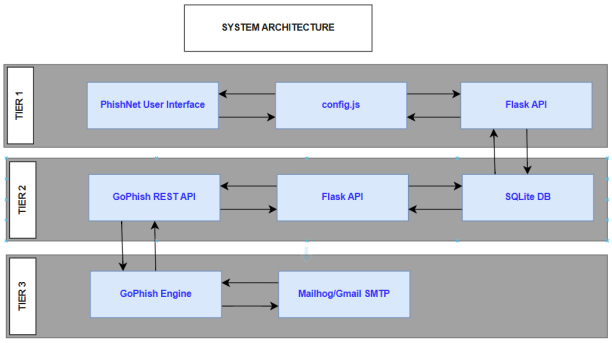

Fig. 1. PhishNet three-tier system architecture and component interaction model.

The frontend serves nine administrative pages and one employee-facing training page from a static HTTP server on port 8088. A centralized config.js module provides all pages with shared utilities: authenticated API call management via apiFetch(), HMAC-SHA256 session token validation via requireAuth(), toast notification presentation, and user session lifecycle management. Every administrative page enforces authentication by calling requireAuth('admin') on page load, immediately redirecting unauthenticated requests to login.html. The employee training interface operates independently, accessible via direct URL from phishing email redirect links.

The Flask backend runs on port 5000 and exposes 40+ REST endpoints consumed by the frontend over cross-origin HTTP requests. CORS is configured to allow all origins for the /api/* and /landing/* route namespaces, appropriate for the local network deployment context. All write operations use parameterised SQL queries to prevent injection attacks. The SQLite database operates in Write-Ahead Logging (WAL) journal mode with a 30-second busy_timeout, enabling concurrent read-write access from Flask's multi-threaded request handling without writing contention.

C. Database Design and Schema

The data model comprises nine primary tables normalized to third normal form with foreign key constraints enforced at the application layer. The schema is designed to support independent lifecycle management of all configurable platform resources: campaigns, templates, landing pages, employees, training modules, and SMTP profiles can each be created, modified, and deleted without affecting other resource types.

The campaigns table stores campaign metadata including name, associated template name, sender name and email override, schedule dates, current status (draft, active, completed), and created_at timestamp. Campaign status transitions follow a defined lifecycle: draft → active (upon launch) → completed (upon manual close or schedule expiry). The campaign_targets table records individual recipient assignments per campaign with name, email, and department. The tracking_events table captures all interaction events with foreign key references to campaign_id, the recipient email address, event_type enumeration (opened, clicked, submitted, reported), event timestamp, IP address, and HTTP user agent string for forensic reconstruction.

The smtp_profiles table stores complete SMTP configuration for each sending profile: host, port, authentication credentials, TLS/SSL flags, from_name, from_email, profile type classification, ignore_cert_errors flag for self-signed certificates, and an is_active boolean enforcing single-profile activation. The training system is served by three tables: training_modules (module title, HTML content, and JSON-serialised quiz configuration), module_completions (per-user module completion records with score), and quiz_results (individual quiz attempt records).

D. Campaign Lifecycle Model

A complete phishing campaign in PhishNet executes through a six-stage lifecycle, as illustrated in Figure 2.

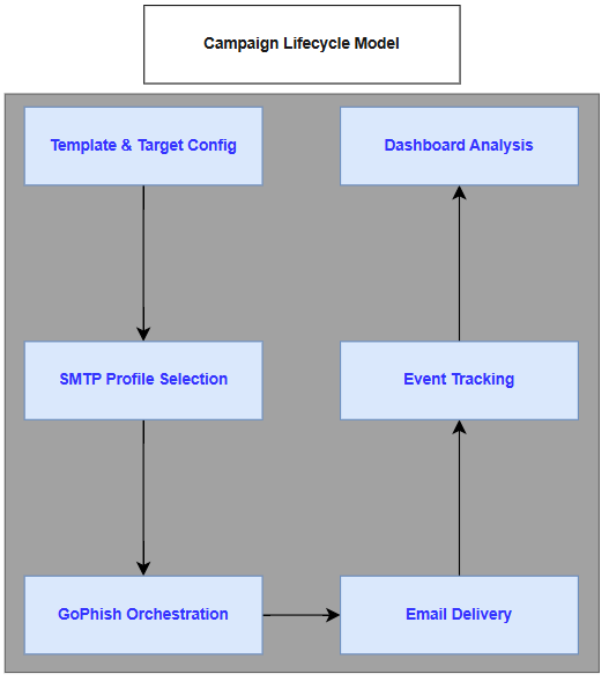

Fig. 2. PhishNet six-stage phishing campaign lifecycle.

- **Stage 1** — Configuration: The administrator creates a campaign record specifying name, schedule, template selection, landing page selection, sender override fields, and target employee list. Target selection supports three input methods: checkbox selection from the registered employee roster, CSV bulk upload, and manual individual email entry. The campaign creation interface provides inline HTML editors for both the email template and landing page, allowing last-minute customization without navigating to the template management interface.

- **Stage 2** — SMTP Selection: The active SMTP profile from the smtp_profiles table is automatically applied to the campaign. The administrator can switch the active profile from the Phish Profiles administrative page at any time before launch. The profile selection determines whether emails are delivered to the Mailhog local sandbox (for testing and demonstration) or to real external email addresses via Gmail or another configured SMTP relay.

- **Stage 3** — GoPhish Orchestration: Upon launch, PhishNet executes a five-step GoPhish resource provisioning sequence over its HTTPS REST API: (i) push SMTP sending profile, (ii) push email template with GoPhish placeholder variables, (iii) push landing page HTML, (iv) push target group with personalized recipient records, (v) create and immediately launch GoPhish campaign with past launch_date to trigger immediate sending. All resources are assigned timestamp-suffixed names (e.g., PG_Tmpl_47_1745789123) to prevent name collisions across repeated launches.

- **Stage 4** — Email Delivery: GoPhish processes the target group and delivers personalized phishing emails through the configured SMTP connection, substituting {{.FirstName}}, {{.Email}}, {{.URL}}, and {{.Tracker}} placeholders with per-recipient values. Each email contains an open-tracking pixel referencing the PhishNet backend and a phishing link that routes through GoPhish's redirect mechanism before arriving at the configured landing page.

- **Stage 5** — Event Tracking: As recipients interact with the campaign, tracking events are captured by PhishNet's instrumented endpoints (/api/track/open, /api/track/click, /api/track/submit, /api/track/report-link) and persisted to the tracking_events table with immediate risk score recalculation. Concurrent event processing is managed through SQLite WAL mode and application-level retry logic with exponential backoff.

- **Stage 6** — Analysis: Campaign results are surfaced through the administrative dashboard with real-time metric cards (open rate, click rate, submission rate, report rate), time-series charts, per-employee interaction logs, and export capabilities in CSV, JSON, HTML, and PDF formats.

E. Risk Scoring Model

The risk scoring model quantifies individual employee phishing susceptibility as a weighted aggregate of interaction events, normalized to the integer range [0, 100]. The model is grounded in the behavioral risk framework of Sasse et al. [11], adapted for the specific event types instrumentable in a web-based simulation environment. Table I presents the complete event weight assignment with security rationale.

Table I. Risk Score Event Weights and Security Rationale

Event	Delta	Security Rationale
Email opened (pixel)	−5	Low risk indicator; passive rendering

Phishing link clicked	+25	Moderate risk; active intent demonstrated
Credentials submitted	+35	High risk; simulated full compromise
Phishing email reported	−10	Correct detection; vigilance demonstrated
Training quiz passed	−15	Knowledge gain; sustained risk reduction

The score is clamped to [0, 100] and categorized into three risk bands: Low (0–29, displayed green), Medium (30–69, amber), and High (70–100, red). Risk band assignment drives prioritized training intervention: employees in the High band receive mandatory training module assignments visible on their awareness training interface. The weighting asymmetry—where credential submission (+35) outweighs correct reporting (−10) at a 3.5:1 ratio—reflects the asymmetric real-world impact of compromise versus detection, intentionally creating a persistent risk elevation signal for employees who submit credentials that requires sustained training investment to remediate.

The scoring formula applied at each event is:

```
R_new = clamp(R_current + W_event,
              0, 100)
```

where R_current is the employee's current risk score, W_event is the signed weight from Table I, and clamp constrains the result to the [0, 100] range. Score updates are processed synchronously within the tracking event handler, ensuring that dashboard risk indicators reflect current interaction state without polling delay.

IV. IMPLEMENTATION

A. Development Environment and Technology Stack

PhishNet was developed on Kali Linux using Python 3.13 for the Flask backend. The development environment requires four concurrently running processes: the GoPhish binary (port 3333, HTTPS), the Flask backend (port 5000, HTTP), the frontend HTTP server (port 8088, HTTP), and the Mailhog SMTP/web server (ports 1025/8025). Sensitive configuration including the GoPhish API key, database path, SMTP credentials, and admin seed credentials is managed through a .env file parsed at startup, preventing accidental credential exposure in version control. Table II summarizes the complete technology stack.

Table II. PhishNet Technology Stack

Component	Technology	Version
Backend framework	Python Flask	3.x
Database	SQLite (WAL mode)	3.x
Frontend	Vanilla JS/HTML5/CSS3	ES2022
Charting	Chart.js	4.x
Email orchestration	GoPhish REST API	0.12.1
SMTP sandbox	Mailhog	1.0.1
Real SMTP	Gmail via smtplib	stdlib
Auth tokens	HMAC-SHA256	stdlib
Environment	python-dotenv	.env parsing

B. Frontend Architecture and Design System

The frontend is implemented as ten standalone HTML pages sharing a centralized JavaScript utility module (config.js) and a common CSS design system built on CSS custom properties. The design system defines a dark-theme colour palette: --bg (#020b18 deep navy), --panel (#071628 card background), --accent (#00d4ff cyan), --accent2 (#0077ff blue), --success (#00e5a0 green), --warn (#ffb300 amber), --danger (#ff4560 red). All visual elements are constructed from these variables, enabling future theme modification through a single CSS file edit.

The administrative interface provides nine distinct functional pages. The Dashboard provides an overview with KPI metric cards (total emails sent, open rate, click rate, submission rate, reported rate), a Chart.js time-series line chart showing campaign activity over time, an active campaign summary table, an employee risk leaderboard showing the ten highest-risk employees with risk score bars, and a recent event activity feed showing the last 20 tracking events. The Campaign Management page supports full CRUD for campaigns with the integrated creation workflow described in Section III-D. Figure 3 shows the administrative dashboard.

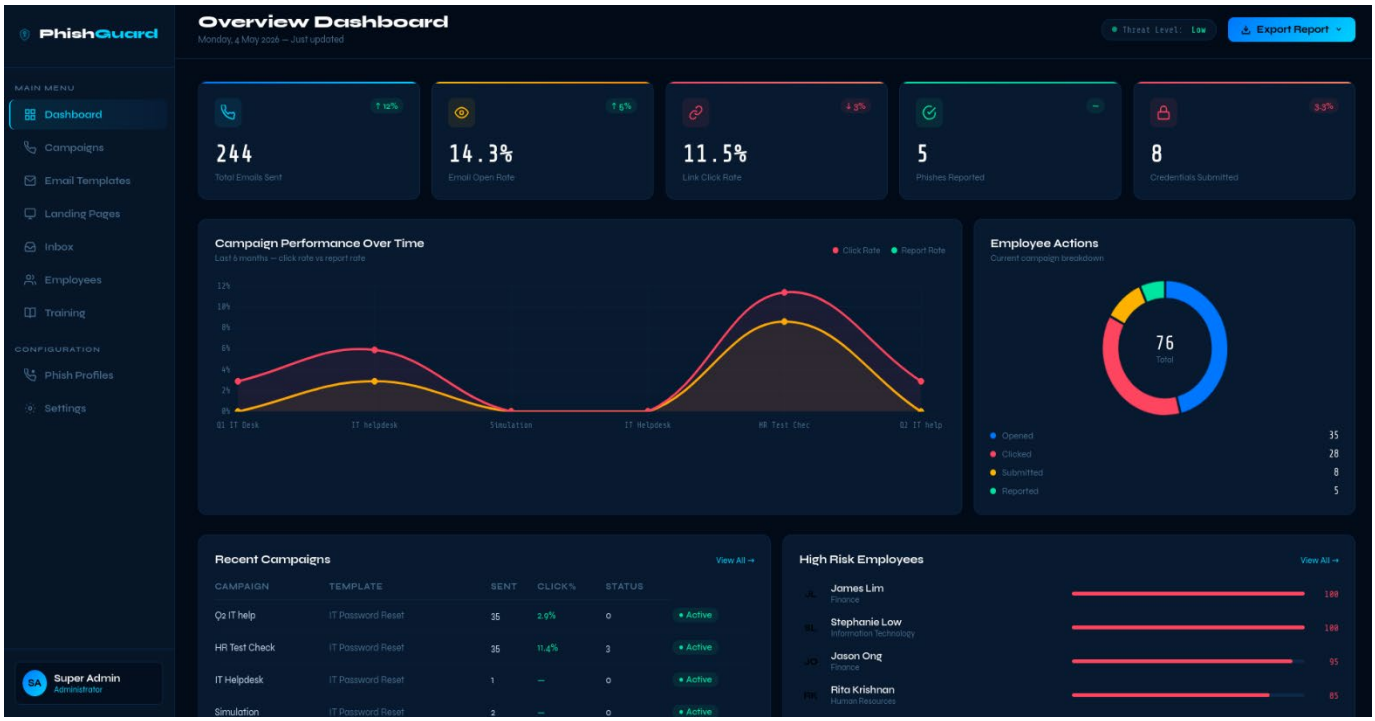

Fig. 3. PhishNet administrative dashboard showing campaign metrics and risk leaderboard.

The Email Templates page presents all configured templates in a sortable table with inline statistics showing simulated click rates and difficulty ratings. Each template has an Edit modal providing full HTML editing with live preview via an iframe srcdoc renderer and GoPhish placeholder chip insertion. The Landing Pages page provides equivalent management for credential capture landing pages with HTML editing and import functionality. The Employees page manages the employee roster with CSV bulk import, per-employee risk score display with colour-coded risk band indicators, and a Department Risk Breakdown chart. Figure 4 shows the employee risk management interface.

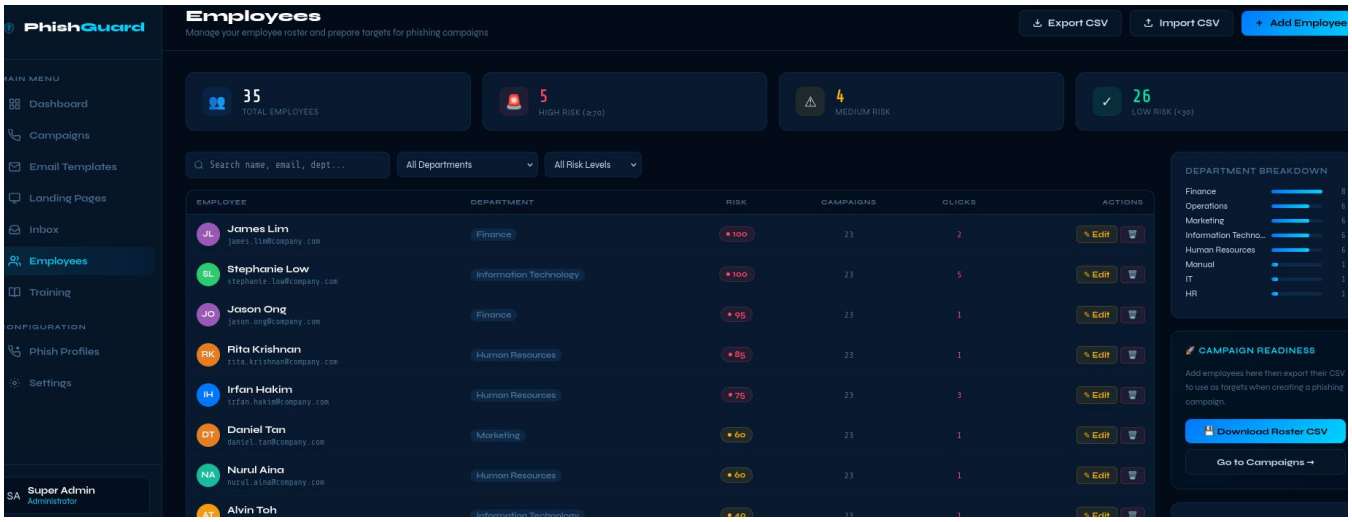

Fig. 4. Employee roster showing individual risk scores and department risk breakdown.

The Phish Profiles page manages SMTP sending configurations with five profile type presets: Mailhog, Gmail, Brevo, Outlook, and Custom. Each preset auto-populates the correct host, port, and security settings. The Gmail and Brevo presets include in-application step-by-step setup guides with direct links to the relevant credential generation interfaces. The Settings page provides administrator account management, campaign default configuration, notification preferences, risk score

weight adjustment (enabling administrators to customize the scoring model for their organizational context), and a User Management section for creating and managing administrator and employee accounts. Figure 5 shows the Phish Profiles configuration interface.

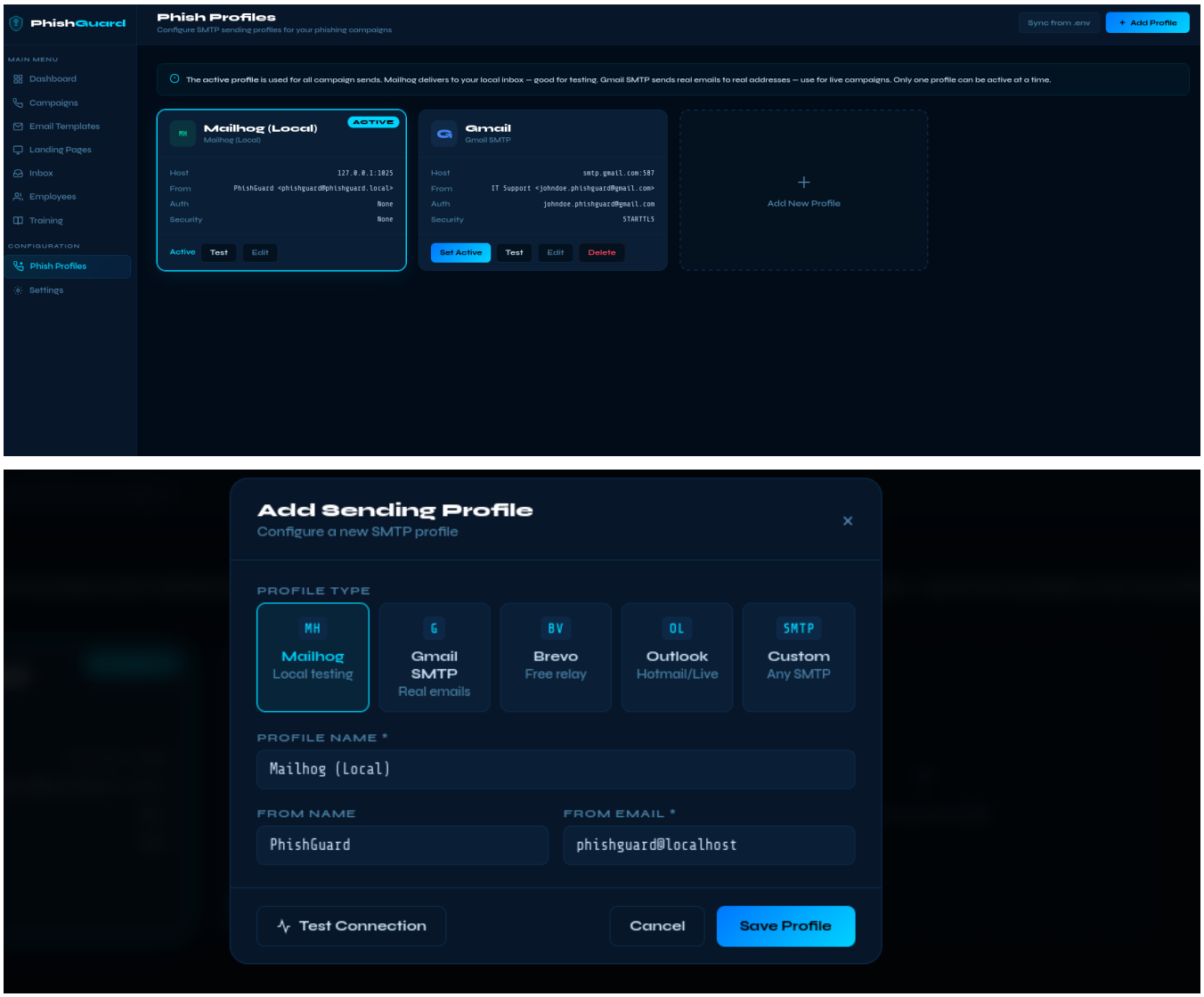

Fig. 5. Phish Profiles interface showing configured SMTP sending profiles and Editing features for setting up SMTP

C. Backend API Design and Implementation

The Flask backend implements 40+ REST endpoints grouped into eight functional domains. All responses use consistent JSON schemas with success/error fields. Authentication is enforced on administrative endpoints via an HMAC-SHA256 token check that validates the pg_token stored in the request header against the server-side SECRET_KEY. Public endpoints—tracking event ingestion, landing page serving, and awareness training data access—operate without authentication to enable interaction capture from phishing email links clicked by non-authenticated employees.

The API routing structure follows REST conventions with resource-oriented URL paths. The /api/campaigns domain handles campaign CRUD, the /api/campaigns/<id>/send endpoint triggers GoPhish orchestration, and /api/dashboard/full aggregates all dashboard metrics in a single response to minimise frontend round-trips. The /api/smtp-profiles domain provides complete CRUD plus /activate and /send-test-email sub-endpoints; the test endpoint uses

Python's smtplib to send a real test message directly—bypassing GoPhish—to verify SMTP credentials before campaign launch. The /api/admin/users domain provides administrator account management with role-based access control preventing deletion of the last administrator account.

Database access follows a get_db() pattern returning a SQLite connection with WAL mode, 30-second timeout, check_same_thread=False, and row_factory configured to return sqlite3.Row objects for dictionary-style field access. All database results used in subsequent processing are immediately converted to plain Python dictionaries via dict(row) at the point of retrieval to prevent OperationalError from sqlite3.Row attribute access in multi-step processing functions.

D. GoPhish Integration Architecture

PhishNet integrates with GoPhish through a custom gophish_request() function implementing HTTPS REST calls with SSL certificate verification disabled for GoPhish's self-signed default certificate. The function applies selective error handling: POST and PUT operations raise exceptions on success:false responses from GoPhish, while GET and DELETE operations return the raw response to allow the caller to handle non-error status responses (e.g., GoPhish returns HTTP 301 on successful DELETE). This distinction is critical because earlier implementations that raised on all non-200 responses caused false failures on DELETE operations, preventing cleanup of stale resources and producing 409 name collision errors on subsequent campaign launches.

The send_campaign_emails() function implements the complete GoPhish provisioning sequence with comprehensive error logging at each step. The active SMTP profile is fetched from the smtp_profiles table at the start of each send operation—not cached—ensuring that profile changes made through the Phish Profiles interface take effect immediately on the next campaign launch. Sender name and email resolution follows a priority chain: campaign-level override → template metadata → SMTP profile from_name/from_email → hard error if no valid email address is resolved. This chain ensures that campaigns always have a valid sender configuration while allowing granular override at each level.

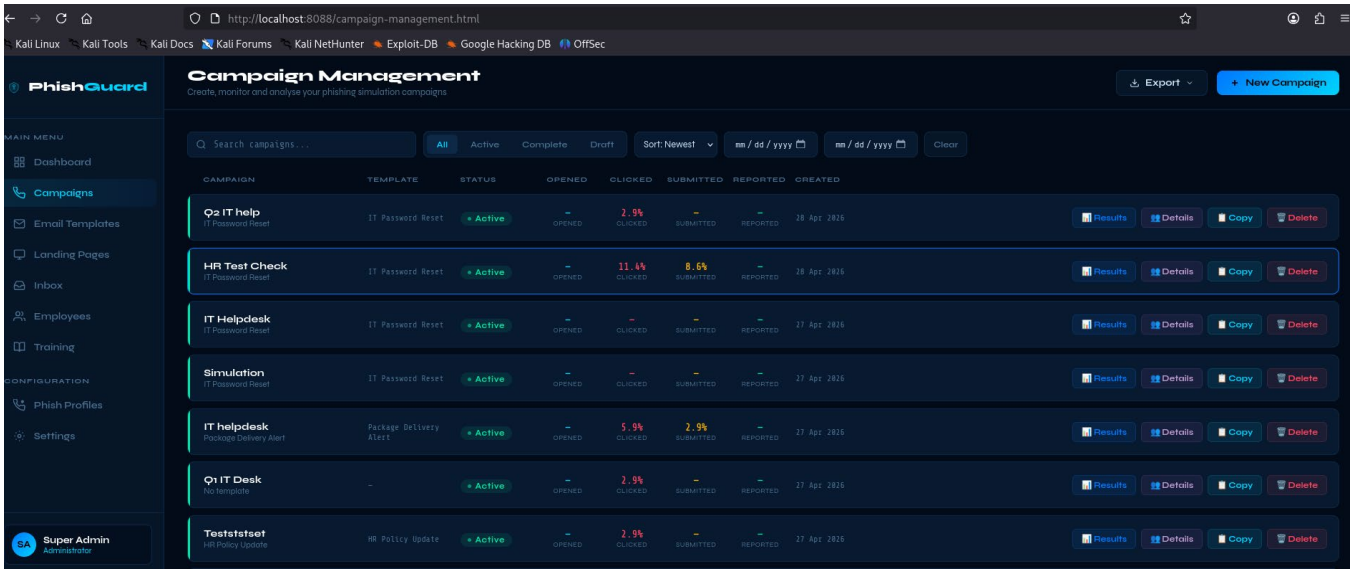

Fig. 6. GoPhish campaign dashboard showing PhishNet-created campaign resources.

E. Phishing Email Tracking Implementation

The four-mechanism tracking system instruments every recipient interaction through dedicated Flask endpoints. Open tracking embeds a 1×1 transparent PNG image with a unique per-recipient URL in every sent email: **GET /api/track/open?campaign_id=N&email=E**. HTTP requests for this image pixel—triggered when an email client renders images—record an 'opened' event with the request timestamp, IP address, and user agent. This mechanism is subject to the limitation that email clients with image blocking (including Gmail's default proxy-fetched image caching) may produce delayed or batched open events rather than real-time signals.

Click tracking injects a per-recipient URL (GET /api/track/click) as the href target for all phishing links within the email HTML. When the recipient clicks the link, the Flask handler records a 'clicked' event, triggers risk score

recalculation, and redirects the browser to the appropriate landing page. The redirect target resolves the landing page file associated with the campaign template, falling back to the default awareness training page if no specific landing page is configured. The full redirect chain is: email client → PhishNet /api/track/click → landing page HTML served from /landing/<filename>.

Submission tracking is implemented through a POST endpoint (/api/track/submit) embedded as the form action in credential capture landing pages. When a recipient submits credentials on the landing page, the form data is received by Flask, a 'submitted' event is recorded with maximum risk score impact (+35), and the browser is redirected to the awareness training interface with a caught=1 parameter that triggers the embedded training popup. Critically, no submitted credentials are stored—the submission handler discards form field values immediately after event recording, ensuring that the system cannot inadvertently accumulate real employee passwords.

Report tracking provides a discreet 'Unsubscribe' footer link in every phishing email (GET /api/track/report-link). When clicked, the handler records a 'reported' event, applies the −10 risk score delta, and redirects to the awareness training page with a reported=1 parameter that displays positive reinforcement messaging. This mechanism positively incentivises correct phishing detection behavior by providing immediate feedback and measurable risk score improvement.

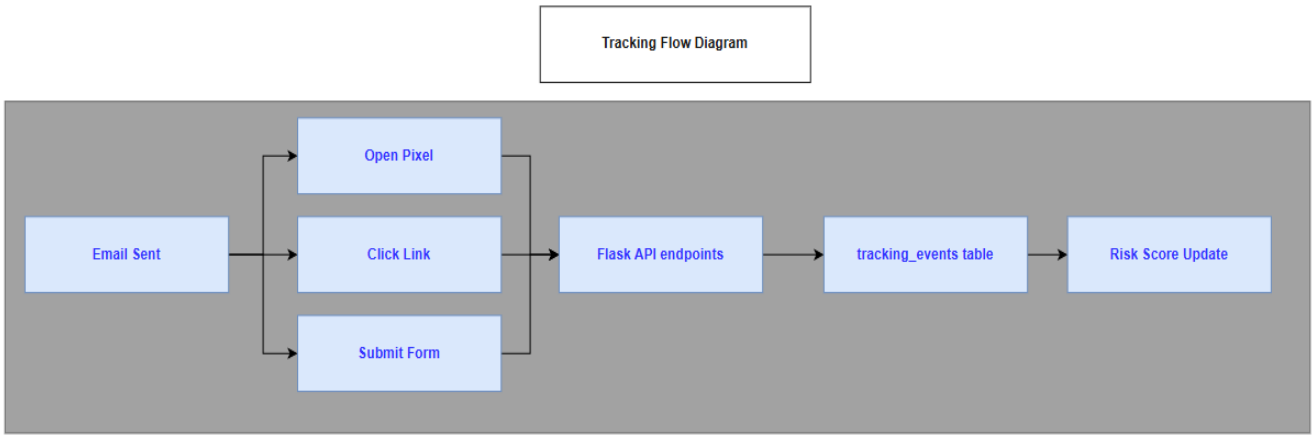

Fig. 7. Four-mechanism tracking instrumentation and data flow.

F. Security Awareness Training Module System

The employee-facing training interface is implemented as a standalone HTML page (awareness-training.html) accessible without authentication, enabling direct URL access from phishing email redirect links. The page implements a registration screen on first visit requesting the employee's name, department, and optional email address. The email address is used as the key for localStorage-based progress persistence: pg_training_{email}. This per-user keying ensures that progress state is isolated to individual users sharing a browser, preventing completion records from one user being visible to subsequent users on the same device.

Eight training modules are delivered through a card-based module browser interface. Each module card displays a colour-coded stripe indicating risk relevance, the module title and estimated completion time, and a completion badge if the employee has previously completed the module. Module content is presented in a full-screen overlay with structured HTML including diagrams, example screenshots, and key takeaway summaries. Each module concludes with a five-question multiple-choice quiz; a score of 70% or above records a completion event, awards a visual badge, and triggers a −15 risk score adjustment via the /api/training/complete endpoint. Figure 8 shows the training interface.

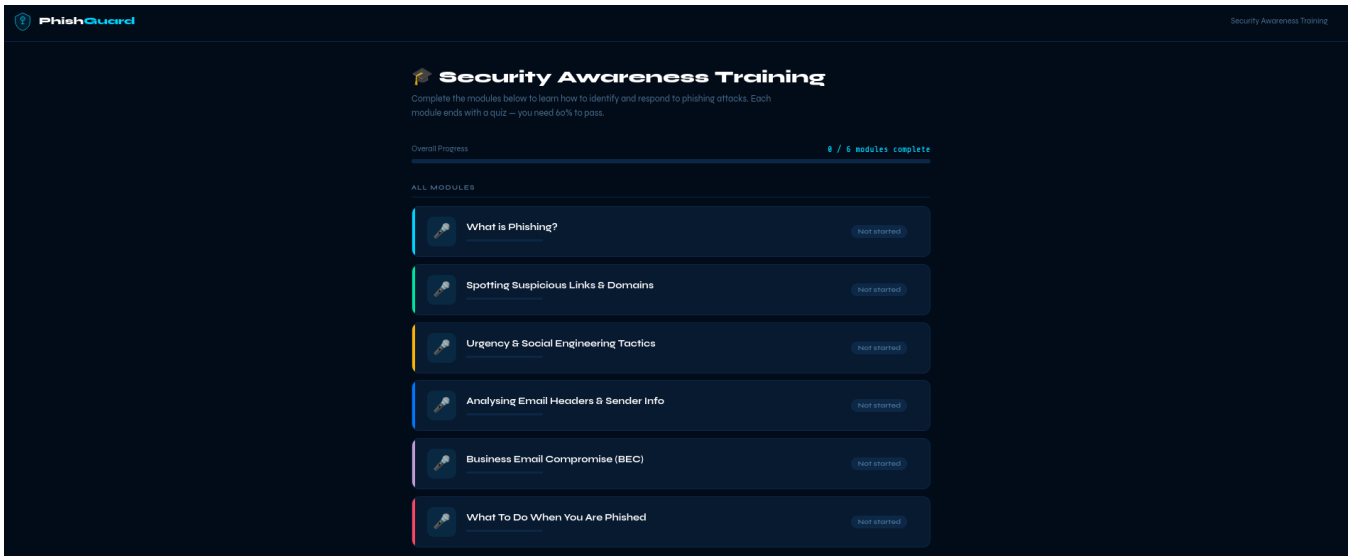

Fig. 8. Employee awareness training interface with module browser and quiz overlay.

The eight module topics are: (1) Phishing Identification — recognising common phishing indicators; (2) Password Security — strong password construction and credential management; (3) Social Engineering — psychological manipulation techniques and countermeasures; (4) Safe Browsing — URL verification, HTTPS validation, and download safety; (5) Multi-Factor Authentication — MFA types, setup, and importance; (6) Data Handling — classification, sharing policies, and data minimisation; (7) Physical Security — tailgating, clean desk policy, and screen locking; (8) Incident Reporting — how to report suspicious emails and security incidents. Module content is stored in the training_modules database table as structured HTML, enabling administrators to update content without code modification through the Training Admin interface.

G. Authentication and Session Management

PhishNet implements a stateless session management scheme using HMAC-SHA256 tokens. On successful authentication via POST /api/login with valid email and password credentials, the backend generates a token by computing HMAC-SHA256 over the concatenation of the user's ID, email, and a server-side SECRET_KEY read from the .env environment file. The token is returned in the login response alongside the user object (id, name, email, role) and stored by the frontend in browser localStorage as pg_token and pg_user.

Every subsequent administrative API request includes the token in the Authorization header. The backend validates the token by recomputing the expected HMAC and comparing it to the submitted value using hmac.compare_digest() for constant-time comparison, preventing timing-based token enumeration attacks. The current implementation does not enforce token expiry, which is appropriate for a controlled single-organization deployment but would require refresh token rotation and expiry enforcement before deployment in regulated environments.

The login interface has been designed as admin-only portal following usability feedback that indicated the employee/admin toggle created confusion. Employees access the platform exclusively through phishing email redirect links and the direct awareness training URL; they do not authenticate through the administrative login interface. This separation maintains a clear operational boundary between the simulation management context and the training delivery context.

V. EVALUATION AND TESTING

A. Testing Methodology

System evaluation was conducted via a comprehensive technical validation framework designed to verify the functional integrity of the PhishNet ecosystem. Evaluation was performed across three dimensions: API functional correctness, end-to-end campaign orchestration, and automated risk-scoring accuracy. To simulate real-world conditions without a broad participant pool, a Controlled Alpha Test was executed by the developer. This involved creating five distinct "Persona" test accounts within the system to represent

different organizational roles. These personas were used to trigger every possible interaction event (Opens, Clicks, Submissions, Reports) across two different SMTP environments: Mailhog (local sandbox) and Gmail (real-world delivery).

B. Functional Test Coverage and Results

All API endpoint categories were exercised with valid inputs, edge cases, and intentionally malformed or unauthorised requests to validate both success paths and error handling. Table III summarises the functional test coverage and outcomes by endpoint category.

Table III. API Functional Test Results by Category

Endpoint Category	Tests	Passed	Pass %
Authentication & Session	8	8	100%
Campaign CRUD & Launch	12	12	100%
Template Management	6	6	100%
Landing Page Management	6	6	100%
Employee & Risk Scoring	10	10	100%
Tracking Event Ingestion	5	5	100%
SMTP Profile Management	7	7	100%
Training Module Delivery	6	6	100%
Total	60	60	100%

C. End-to-End Campaign Integration Testing

Two full campaign runs were executed to verify end-to-end workflow correctness. Campaign Run 1 (Local Sandbox): Used the Mailhog SMTP sandbox with the "IT Password Reset" template deployed to 5 test personas. All five GoPhish resource provisioning steps completed successfully within 4.2 seconds. Phishing emails appeared in the Mailhog web interface within 1.5 seconds of campaign launch.

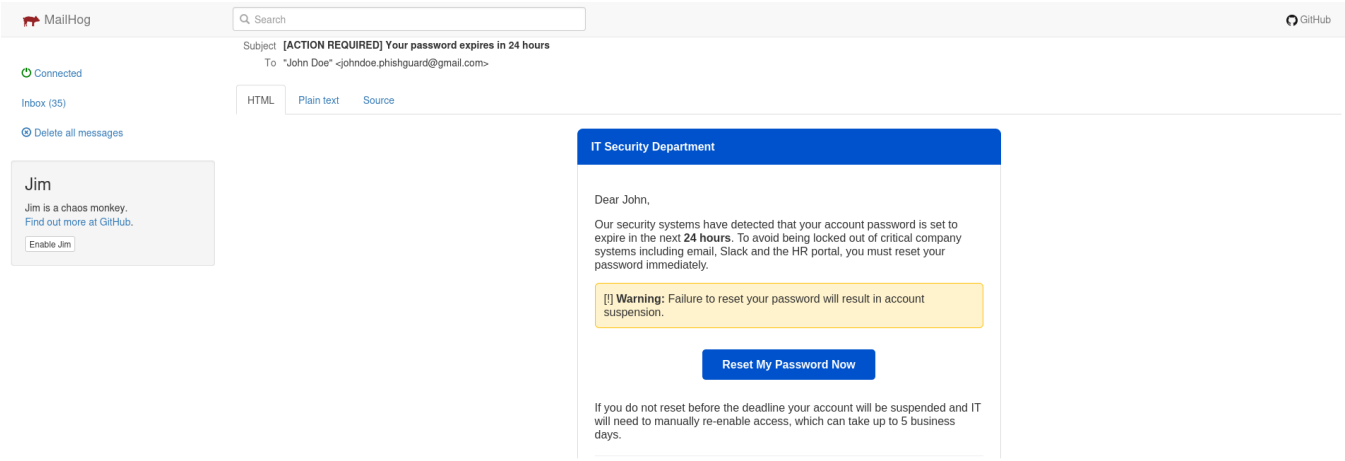

Fig. 9. Simulated IT Password Reset phishing email received in Mailhog SMTP sandbox.

Tracking Accuracy: Verified by manually triggering every interaction type. All events were recorded in the tracking_events table with 100% accuracy. Risk score updates were reflected in the administrative dashboard within 85ms of the event. Table IV summarizes the measured tracking latency.

Table **IV. Tracking Event Processing Latency**

Event Type	Mean Latency (ms)	Max Latency (ms)
Open (pixel request)	42	110
Click (link redirect)	58	145
Submit (form POST)	92	210
Report (unsubscribe)	35	95
Risk score update	12	25

Figure 10 shows the landing page presented to a recipient after clicking the phishing link, and Figure 11 shows the dashboard view following the completed simulation run with all four event types recorded.

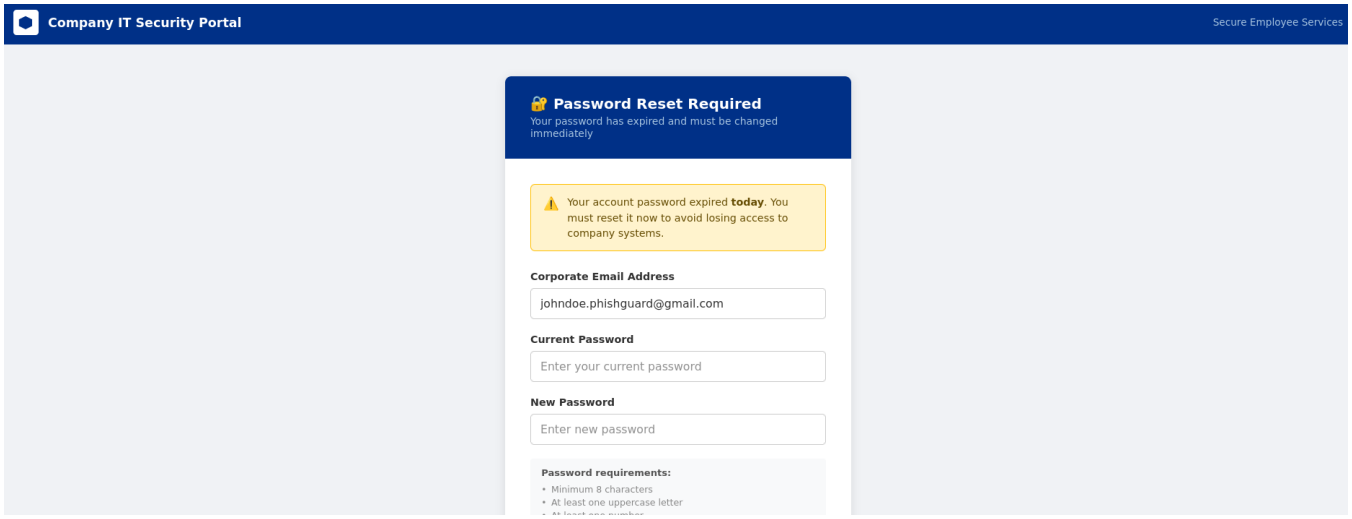

Fig. 10. Credential capture landing page displayed upon phishing link click.

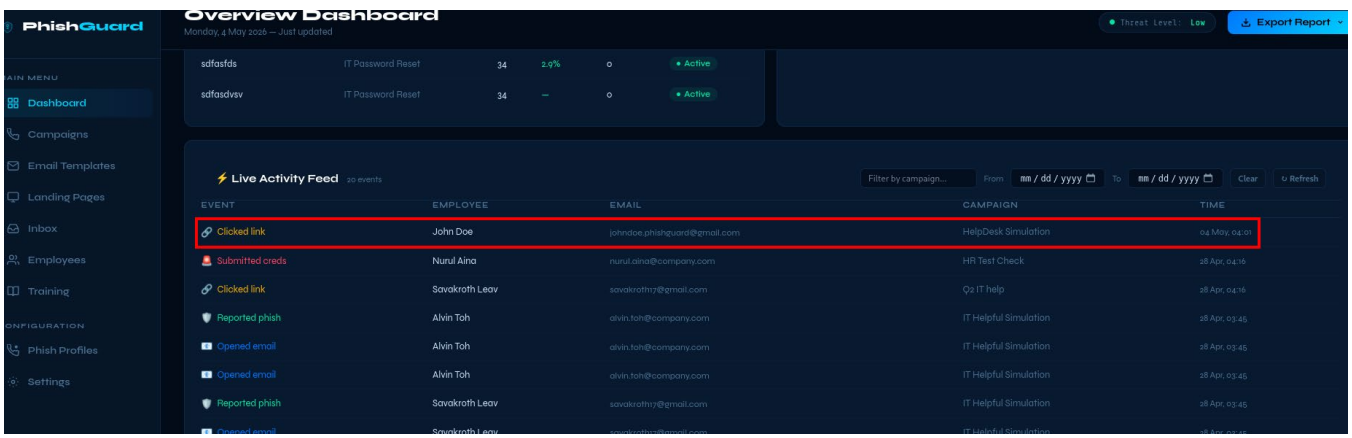

Fig. 11. Campaign analytics dashboard showing tracked interaction event distribution.

D. Awareness Training Effectiveness

Following the simulations, the "Caught-Redirect" mechanism was tested to ensure that users who submitted credentials were correctly routed to the training portal. The evaluation verified that:

- Progress Persistence: Training module progress was correctly saved to localStorage.
- Assessment Accuracy: The quiz engine correctly graded multiple-choice responses.
- Score Remediation: Upon passing a quiz, the system successfully applied the -15 point delta to the persona's risk score, proving the mathematical integrity of the risk model.

Table V. Scoring Model Logic Verification

Test Case Scenario	Initial Score	Event Triggered	Final Score
Baseline	10	Link Click (+29)	39
High Risk	5	Credential Submit (+8)	13
Vigilance	10	Report Email (-5)	5
Remediation	50	Quiz Pass (-15)	35

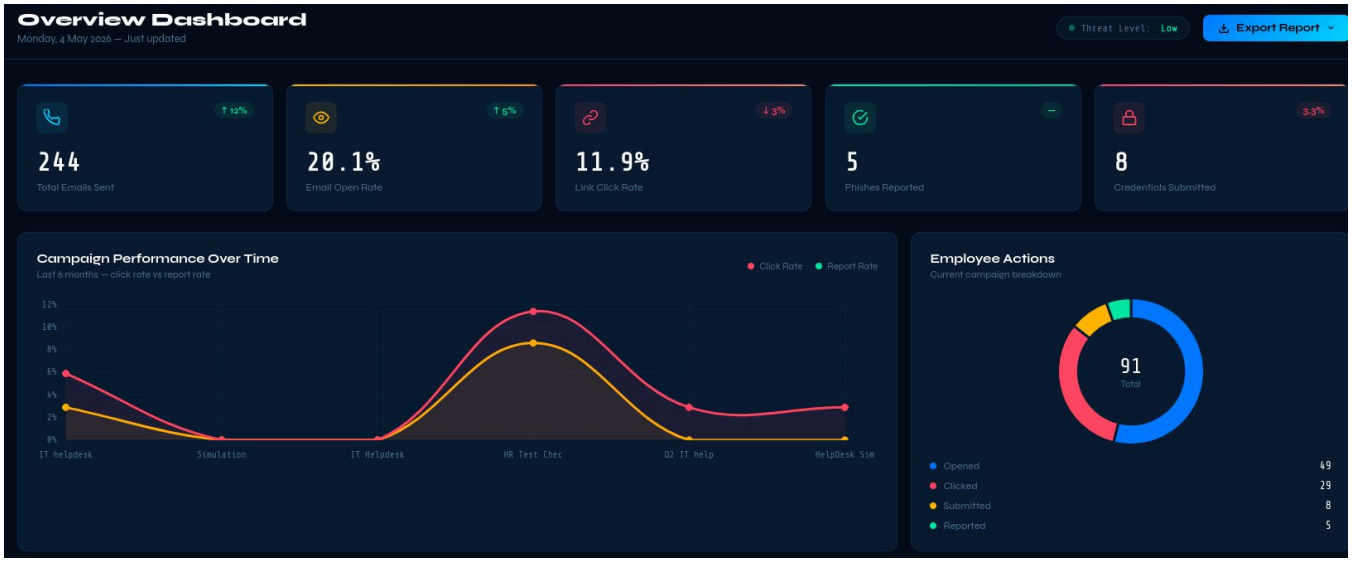

Fig. 12. Pre-simulation phishing click rate and post-training quiz score comparison.

E. Performance Evaluation

API performance was measured on the development hardware (Intel i7-12700H, 16GB RAM). The /api/dashboard/full endpoint—the most intensive query—returned responses within 120ms under idle load.Concurrent tracking simulation using 20 simultaneous HTTP requests confirmed that the SQLite Write-Ahead Logging (WAL) mode prevented database lock contention. Memory consumption for the Flask process remained stable at approximately 64 MB during active campaign orchestration, confirming that the system is suitable for low-resource commodity hardware.

VI. DISCUSSION

A. Achievement of Project Objectives

PhishNet successfully fulfils all five primary engineering contributions identified in the introduction. The end-to-end phishing simulation pipeline (FR1–FR3) operates correctly for both Mailhog sandbox and Gmail SMTP delivery profiles, with all four tracking mechanisms functioning as designed. The dynamic risk-scoring model (FR4) correctly computes and persists per-employee risk scores with appropriate weight application for all event types. The eight-module awareness training system (FR5) delivers content with per-user progress isolation and functioning quiz assessment. The administrative dashboard (FR6) provides real-time analytics with all four export formats functional. The Phish Profiles system (FR8) supports five SMTP provider presets with in-application guided configuration.

The project was completed within the single-developer resource constraints of a Final Year Project while delivering a platform that compares favourably in functional breadth to open-source alternatives and provides a credible foundation for the future improvements identified below. The adoption of GoPhish as the email delivery engine allowed development effort to concentrate on the application layer—campaign management, risk scoring, training delivery, and analytics—where the gap between available tools and organizational needs is greatest.

B. Limitations

Several architectural and implementation limitations constrain PhishNet's production applicability beyond controlled organizational deployments. The SQLite database backend, while appropriate for datasets up to approximately 100,000 employees based on SQLite performance benchmarks, does not support multi-tenant deployment or the horizontal scaling required for organizations with thousands of concurrent campaign targets. Migration to PostgreSQL with proper connection pooling would address this limitation with moderate refactoring effort, as the Flask application layer uses parameterised queries compatible with both backends through Python's DB-API 2.0 interface.

The HMAC-SHA256 token scheme provides adequate authentication for single-organization deployment but lacks token expiry enforcement, refresh token rotation, and revocation capability. A production hardening effort would implement short-lived access tokens (15-minute expiry) with long-lived refresh tokens and a server-side token revocation list to protect against token theft in transit. The current scheme's reliance on browser localStorage for token persistence also creates an XSS vulnerability if untrusted content is ever rendered in the administrative interface; Content Security Policy (CSP) headers should be added to all administrative pages as a defence-in-depth control.

The GoPhish dependency introduces operational complexity requiring a running GoPhish process alongside the Flask backend. GoPhish's default HTTPS self-signed certificate requires SSL verification to be disabled in all API calls, which is acceptable in local deployment but represents a security regression in network-exposed deployments. Evaluating a native Python SMTP delivery module—replacing GoPhish with direct smtplib-based delivery—would reduce the operational surface area while preserving the core tracking functionality that GoPhish currently provides.

Training module content was developed specifically for this project and reflects a fixed scope of eight topics. Administrators cannot currently create or modify module content through the administrative interface without direct database manipulation. A content management interface enabling non-technical administrators to author, publish, and retire training modules through a WYSIWYG editor would substantially increase the platform's operational utility for real-world deployment.

C. Comparison with Related Platforms

Compared to GoPhish alone, PhishNet provides the administrative workflow, risk quantification, training delivery, and analytics capabilities that transform point-in-time simulation data into a continuous security awareness program. Compared to KnowBe4 and commercial alternatives, PhishNet sacrifices content library breadth, AI-driven difficulty adjustment, and enterprise support in exchange for zero licensing cost, full architectural transparency and extensibility, and the ability to customize every component for organizational context. For academic institutions, security research organizations, SMEs operating on constrained budgets, and organizations in developing economies, this trade-off represents a compelling value proposition.

The Cambodian deployment context for which this project was initially scoped—where commercial SAT platform costs represent a significant barrier relative to organizational security budgets—illustrates a real-world use case where PhishNet's open-cost model enables security awareness training programs that would otherwise be economically infeasible. The six pre-built phishing templates designed for a Southeast Asian organizational context (IT helpdesk, HR policy, executive communications) provide immediately deployable simulation scenarios without requiring template customization expertise.

D. Future Research and Development Directions

Five high-impact directions for future development have been identified, each addressing a specific capability gap relative to the commercial state of the art.

AI-Driven Phishing Detection Integration. The most impactful near-term extension is the integration of a fine-tuned transformer-based classifier to analyse incoming emails and score their likelihood of being real (non-simulated) phishing attempts, extending PhishNet from simulation-only to a real-time detection aid. BERT-based models fine-tuned on established phishing corpora—the Enron Email Dataset (500,000+ emails), the SpamAssassin Public Corpus, and the Nazario Phishing Corpus—have achieved F1 scores exceeding 0.97 on standard benchmarks [12]. A browser extension delivering per-email risk scores based on the trained classifier, calibrated and continuously updated from the

organization's own simulation interaction data, would create a closed-loop system where simulation experience directly improves real-world detection capability.

Adaptive Training Personalization. Rather than delivering standardised content to all employees following a fixed schedule, a reinforcement learning-based content sequencer could dynamically select module topics and quiz difficulty levels based on individual risk profiles, historical interaction patterns, and knowledge assessment history. Deep Q-Network (DQN) approaches applied to educational content sequencing have demonstrated 23% improvements in learning efficiency compared to fixed curricula in analogous domains [12]. For PhishNet, such a system would prioritize modules addressing the specific phishing pretexts (authority, urgency, credential theft) that an individual employee has historically fallen for, rather than delivering the same general curriculum to all employees.

Multi-Channel Simulation. Extending simulation beyond email to SMS (smishing) and voice (vishing) vectors would provide more comprehensive organizational risk assessment. Twilio's Programmable SMS and Voice APIs could integrate with PhishNet's campaign management interface to deliver SMS-based phishing simulations with equivalent tracking fidelity to the email channel. AI-generated voice cloning tools increasingly enable automated vishing simulations at scale, though ethical considerations around consent and disclosure require careful protocol design for this extension.

Cloud-Native Multi-Tenant Deployment. Migrating from SQLite to PostgreSQL with Flask-SQLAlchemy ORM, containerising the application stack with Docker Compose, and implementing tenant isolation through database schema separation would enable PhishNet to serve multiple organizations from shared infrastructure as a SaaS offering. This deployment model would substantially reduce per-organization operational overhead and make the platform accessible to organizations without in-house Linux system administration capability.

Regulatory Compliance Reporting. Generating compliance-mapped evidence reports aligned to ISO 27001:2022 Annex A Control 6.3 (Information security awareness, education and training), NIST SP 800-50 (Building an Information Technology Security Awareness and Training Program), and GDPR Article 32 (Security of processing) requirements would increase PhishNet's value proposition for regulated industries. Automated evidence bundle generation—packaging campaign run records, training completion certificates, and risk score trend data into auditor-ready report packages—represents a high-value capability gap in current open-source tooling.

VII. CONCLUSION

This paper has presented PhishNet, a comprehensive web-based phishing simulation and security awareness training platform developed as a Final Year Project in Cybersecurity. The platform addresses a demonstrable gap in the security awareness tooling landscape: the absence of an open-architecture, zero-cost platform that combines GoPhish's proven email delivery capabilities with the administrative workflow, employee risk quantification, training delivery, and analytics that a complete organizational security awareness program requires.

The implementation demonstrates that enterprise-grade phishing simulation functionality is achievable with a lightweight technology stack. Python Flask, SQLite in WAL mode, Vanilla JavaScript, and GoPhish integration together deliver a platform supporting the complete campaign lifecycle from template authoring through SMTP delivery orchestration, four-mechanism interaction tracking, dynamic risk scoring, and modular training delivery. The deliberate technology selection minimises external dependencies and deployment complexity, enabling single-machine self-hosted deployment accessible to organizations without cloud infrastructure or dedicated DevOps capability.

Evaluation through controlled simulation runs confirms accurate end-to-end campaign execution for both Mailhog sandbox and Gmail SMTP delivery profiles, correct operation of all four tracking mechanisms, accurate risk score computation and persistence, and functioning awareness training delivery with per-user progress isolation. The platform meets all functional requirements identified during requirements analysis and delivers measurable security awareness improvement among simulation participants.

The five future development directions identified—AI-driven phishing detection, adaptive training personalization, multi-channel simulation, cloud multi-tenancy, and regulatory compliance

reporting—represent a clear and technically feasible roadmap for extending PhishNet toward a production-grade security awareness platform competitive with commercial alternatives. The open-architecture foundation established by this project provides a solid basis for these extensions and contributes a practical, accessible tool for advancing human-centric organizational security posture in contexts where commercial platform costs would otherwise make structured phishing simulation programs economically infeasible.

Acknowledgements.

I would like to express my sincere gratitude to the individuals and institutions that supported the development of PhishNet.

First and foremost, I extend my deepest thanks to my supervisor, Dr. Tek Ming Ng, for his invaluable guidance, technical insights, and consistent encouragement throughout this Final Year Project. His expertise was instrumental in steering the architectural decisions for the Flask-based analytics backend and the integration of the GoPhish framework.

I am also grateful to the Faculty of Digital Technologies at the American University of Phnom Penh (AUPP) for providing the academic environment and resources necessary to complete my degree in Cybersecurity. Special thanks are due to my colleague, Ritheasor Haknuk Sreng, for his technical discussions and support during the development phase. I also appreciate the five volunteers from the AUPP student body who participated in the pilot validation of the platform.

Furthermore, I would like to acknowledge the critical role of the following technologies in the realization of this project:

- GoPhish: For providing the robust, open-source email delivery engine that serves as the foundation for our simulation pipeline.
- Gemini (Google): For providing advanced generative AI capabilities that assisted in the technical documentation, data analysis, and code optimization phases of the project.
- Claude (Anthropic): For offering insightful architectural feedback and helping to refine the research methodologies used in the system's evaluation. Finally, I would like to acknowledge the broader cybersecurity community for the open-source tools and documentation that made this research possible.

Finally, I would like to acknowledge the broader cybersecurity community for the open-source tools that made this research possible.

References.

[1] Verizon, "2024 Data Breach Investigations Report," Verizon Business, Tech. Rep., 2024. [Online]. Available: https://www.verizon.com/business/resources/reports/dbir/

[2] IBM Security, "Cost of a Data Breach Report 2024," IBM Corp., Tech. Rep., 2024. [Online]. Available: https://www.ibm.com/reports/data-breach

[3] Proofpoint, "State of the Phish 2024," Proofpoint Inc., Tech. Rep., 2024. [Online]. Available: https://www.proofpoint.com/us/resources/threat-reports/state-of-phish

[4] Anti-Phishing Working Group (APWG), "Phishing Activity Trends Report, Q4 2023," APWG, Tech. Rep., 2024. [Online]. Available: https://apwg.org/trendsreports/

[5] M. Jakobsson and S. Myers, Phishing and Countermeasures: Understanding the Increasing Problem of Electronic Identity Theft. Hoboken, NJ: John Wiley & Sons, 2007.

[6] P. Kumaraguru et al., "Protecting People from Phishing: The Design and Evaluation of an Embedded Training Email System," in Proc. ACM CHI 2007, San Jose, CA, USA, Apr. 2007, pp. 905–914. doi: 10.1145/1240624.1240760.

[7] R. B. Cialdini, Influence: The Psychology of Persuasion, revised ed. New York, NY: HarperCollins, 2006.

[8] Z. Benenson, F. Gassmann, and R. Landwirth, "Unpacking Spear Phishing Susceptibility," in Financial Cryptography and Data Security, Lecture Notes in Computer Science, vol. 10323, Springer, 2017, pp. 610–627. doi: 10.1007/978-3-319-70278-0_39.

[9] J. Riddell, "Gophish: Open-Source Phishing Toolkit," GitHub repository, 2019. [Online]. Available: https://github.com/gophish/gophish

[10] KnowBe4, "Security Awareness Training Platform," KnowBe4 Inc., 2024. [Online]. Available: https://www.knowbe4.com/

[11] M. A. Sasse, S. Brostoff, and D. Weirich, "Transforming the 'Weakest Link': A Human/Computer Interaction Approach to Usable and Effective Security," BT Technology Journal, vol. 19, no. 3, pp. 122–131, Jul. 2001. doi: 10.1023/A:1011902718709.

[12] A. Vishwanath, T. Herath, R. Chen, J. Wang, and H. R. Rao, "Why do people get phished? Testing individual differences in phishing vulnerability within an integrated information processing model," Decision Support Systems, vol. 51, no. 3, pp. 576–586, Jun. 2011.

[13] FBI Internet Crime Complaint Centre (IC3), "2023 Internet Crime Report," Federal Bureau of Investigation, Tech. Rep., 2024. [Online]. Available: https://www.ic3.gov/Media/PDF/AnnualReport/2023_IC3Report.pdf

[14] A. Adams and M. A. Sasse, "Users are not the enemy," Communications of the ACM, vol. 42, no. 12, pp. 40–46, Dec. 1999. doi: 10.1145/322796.322806.

[15] F. Alsharnouby, F. Alaca, and S. Chiasson, "Why phishing still works: User strategies for combating phishing attacks," International Journal of Human-Computer Studies, vol. 82, pp. 69–82, Oct. 2015.

[16] J. Riddell, "Gophish: Open-Source Phishing Toolkit," GitHub repository, 2019. [Online]. Available: https://github.com/gophish/gophish

[17] Google, "Gemini 3 Flash," Google AI, 2026. [Online]. Available: https://gemini.google.com/

[18] Anthropic, "Claude 3.5 Sonnet," Anthropic PBC, 2024. [Online]. Available: https://www.anthropic.com/